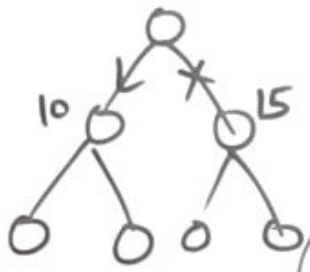
CS 3011
Artificial Intelligence
Credit: 3

By
Debanjan Pathak
Assistant Professor
KIIT University

HEURISTIC IN ARTIFICIAL INTELLIGENCE

'Heuristic in AI' (Rule of thumb) [What, Why, How]

→ It is a technique designed to solve a problem quickly.



8 Puzzle 15 Puzzle 24 Puzzle

$O(b^d)$ 10^{13} 10^{24}

3^{20}

NP → P

0+1+1+2+0+2+2+0+

Manhattan

→ Reduce.

Euclidian distance

$$\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$$

→ good

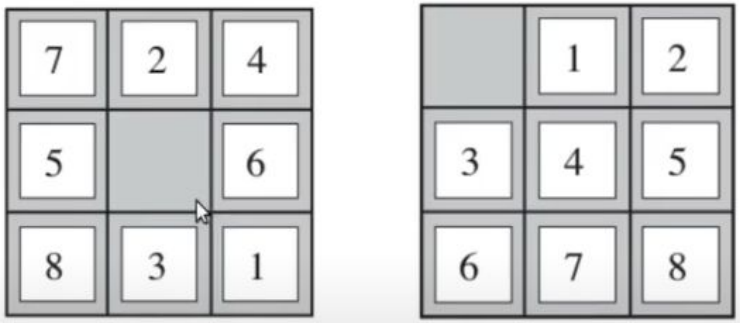
→ optimal

8-Puzzle

Consists of a 3×3 board with eight numbered tiles and a blank space.

A tile adjacent to the blank space can slide into the space.

The objective is to reach a specified goal state.



Start State

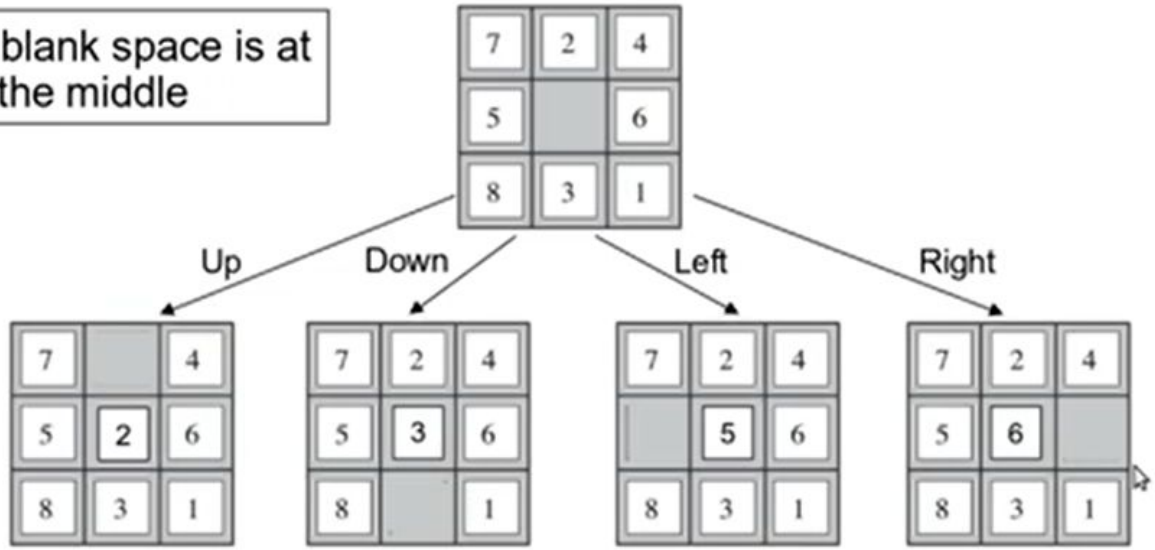
Goal State

Actions in 8-Puzzle

A tile adjacent to the blank space can slide into the blank space.

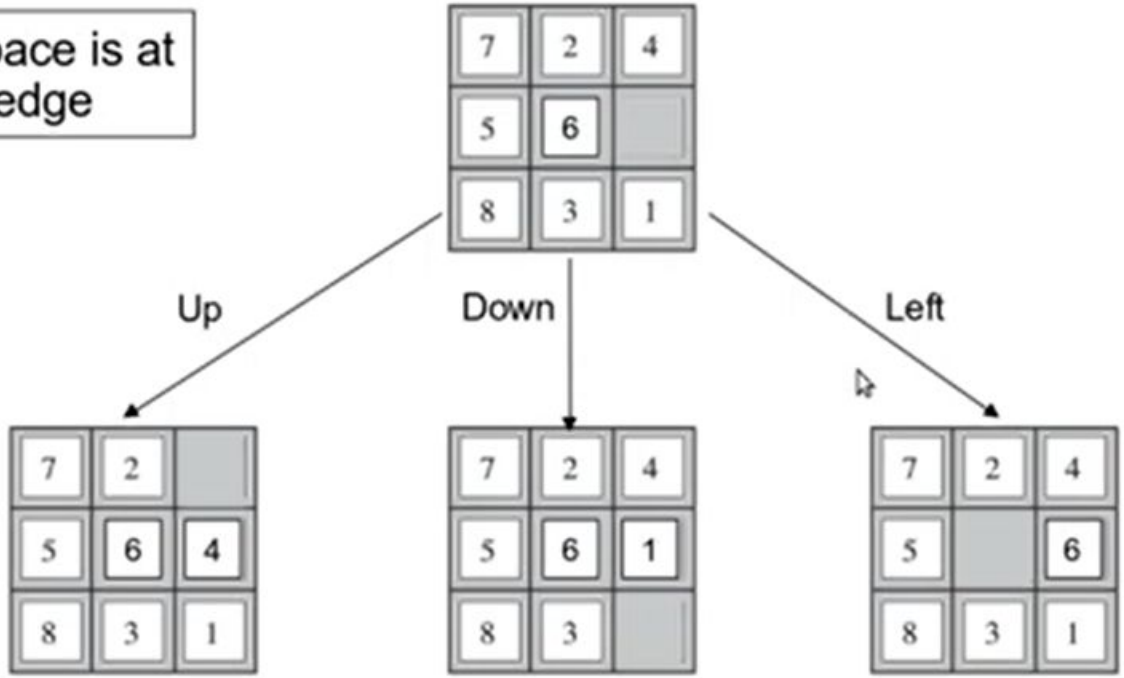
The movement of the adjacent tile can also be thought of the movelemt of the blank space Left, Right, Up, or Down.

Here, blank space is at the middle



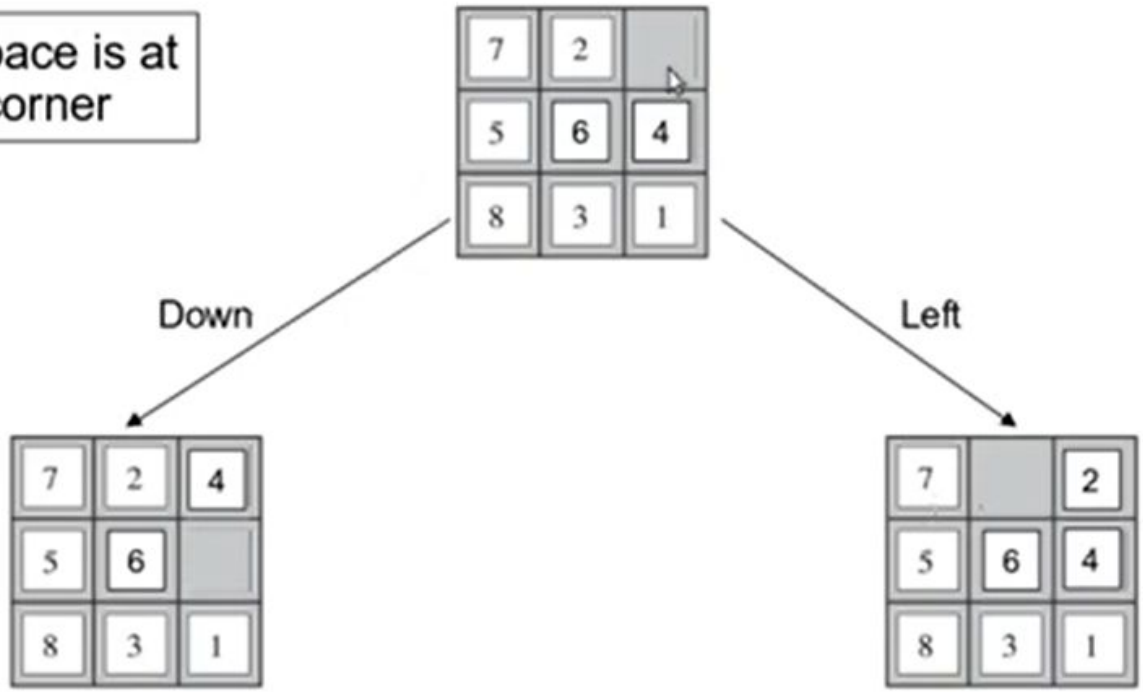
Actions in 8-Puzzle

Here, blank space is at one of the edge



Actions in 8-Puzzle

Here, blank space is at one of the corner



Actions in 8-Puzzle

Blank space is at the middle = 4 actions are possible

Blank space is at one of the edge = 3 actions are possible

Blank space is at one of the corner = 2 actions are possible

Hence, branching factor is about 3

The average solution cost for a randomly generated 8-puzzle instance is about 22 steps.

An exhaustive tree search to depth 22 would look at about $3^{22} \approx 3.1 \times 10^{10}$ states.

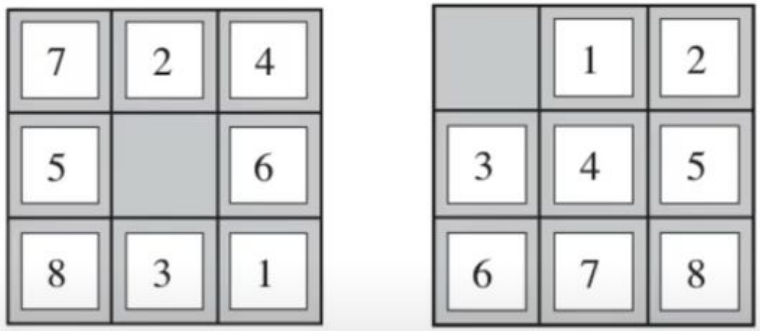
Graph search would look at about 9!/2 = 181,440 states (as only 9!/2 are reachable from a given state).

8-Puzzle: Heuristic 1

h_1 = The number of misplaced tiles

So, h_1 for start state = 8

Each of the tiles is misplaced

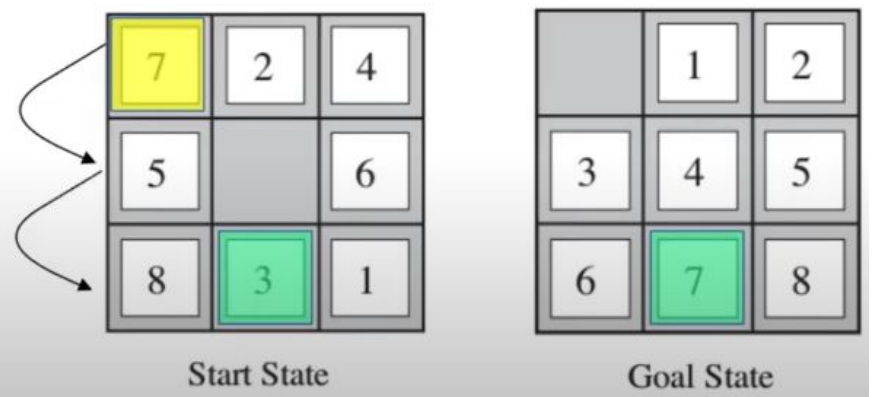


Start State

Goal State

8-Puzzle: Heuristic 2

h_2 = Manhattan distance
= the sum of the horizontal and vertical distances of the tiles from their goal positions



8-Puzzle: Heuristic 2

h_2 for start state = 3 + 1 + 3 + 2 + 2 + 2 + 3 + 2 = 18

Manhattan distance for tile 7 = 3

Manhattan distance for tile 2 = 1

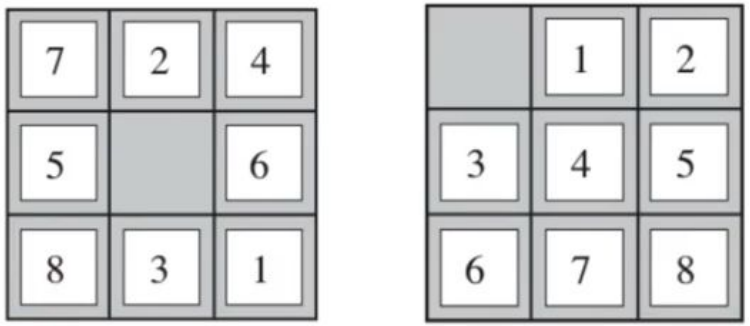
Manhattan distance for tile 1 = 3

Manhattan distance for tile 3 = 2

Manhattan distance for tile 4 = 2

Manhattan distance for tile 5 = 2

Manhattan distance for tile 6 = 3



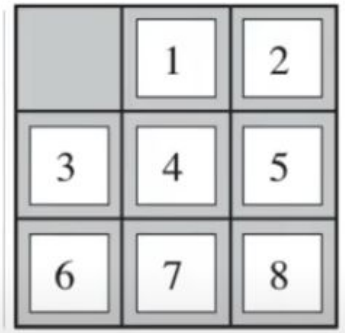
Start State

Goal State

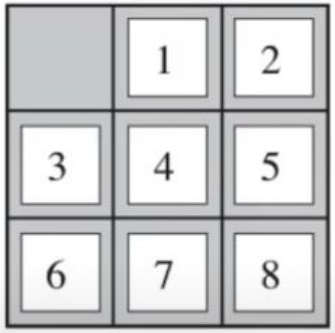
Manhattan distance for tile 6 = 2

Heuristic functions:
nonnegative, with one constraint:
if n is a goal node, then h(n) = 0

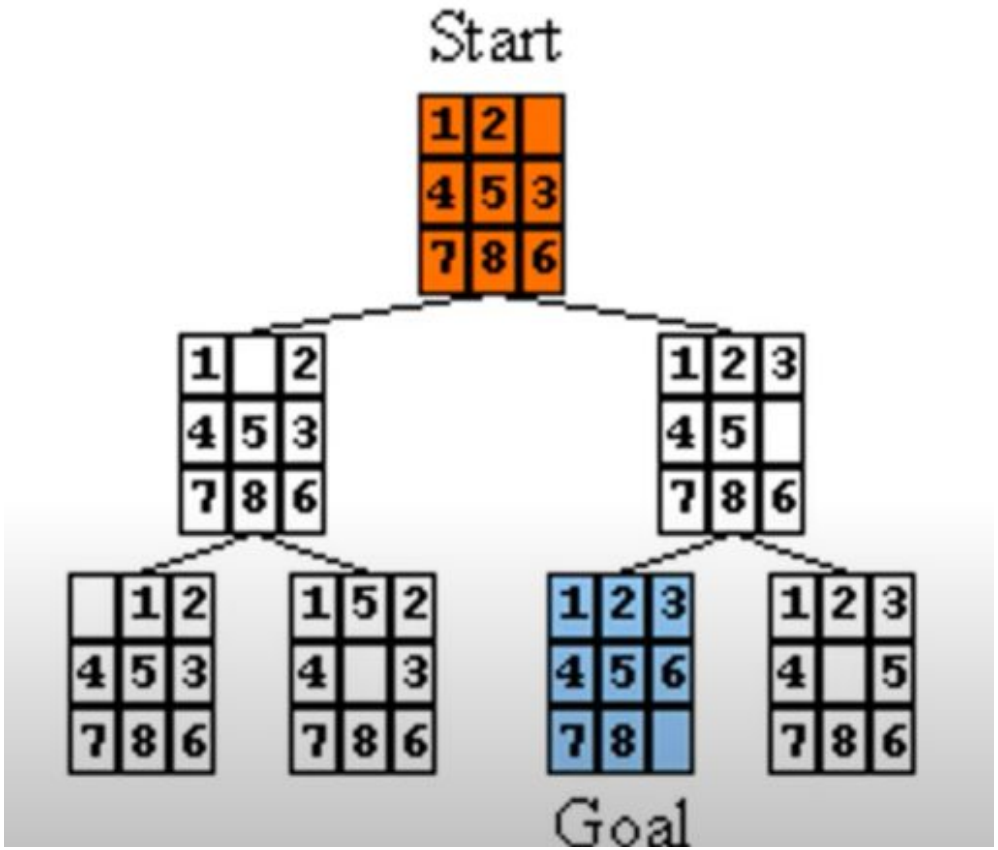
Both h_1 and h_2 satisfy the above constraints



Goal State

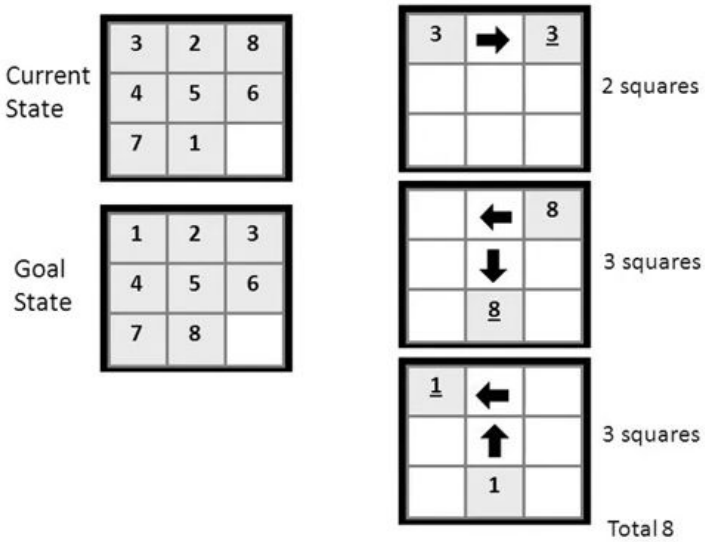


Goal State


Start
Goal

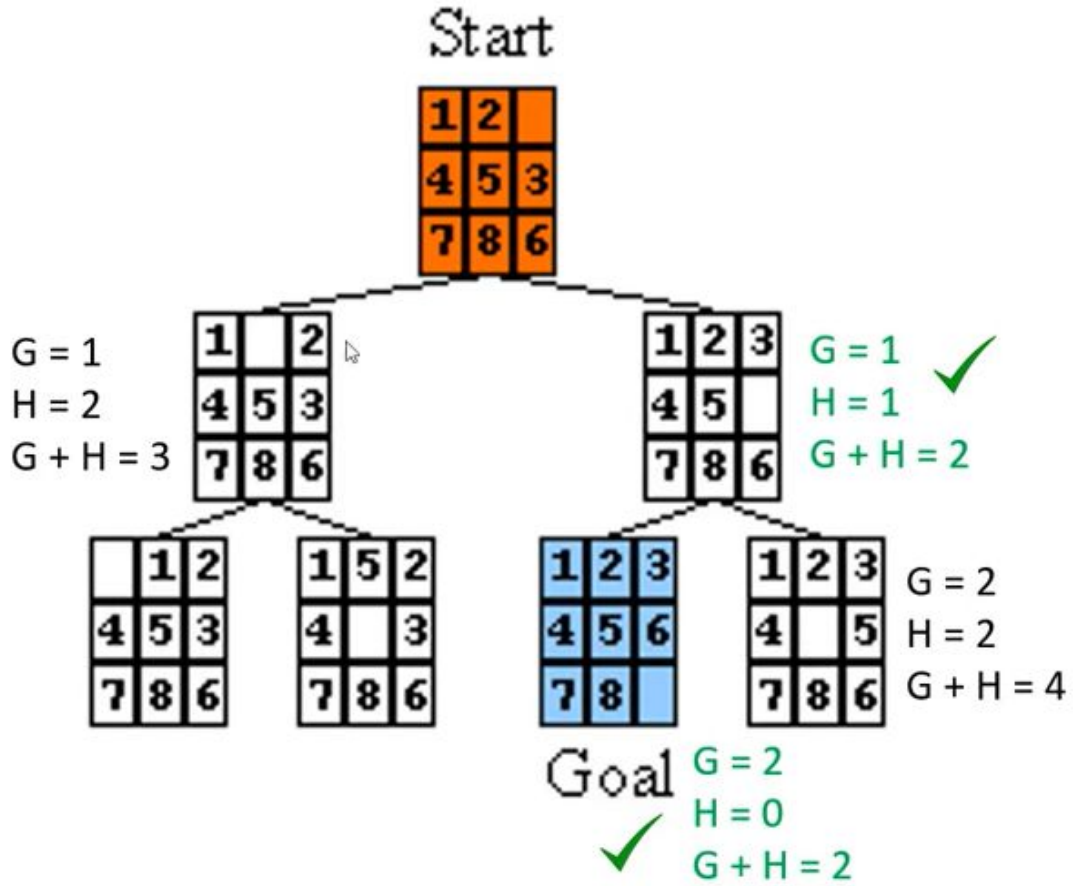
1. G: Cost (Number of steps taken to current state)

2. H: Estimated distance to goal (Estimated number of steps taken from current state to goal state)
   - Manhattan Distance



A* Search

- Identify path with the value (G + H)



Informed Search Algorithms

- So far we have talked about the **uninformed search algorithms** which looked through search space for all possible solutions of the problem without having any additional knowledge about search space.

- But informed search algorithm contains an **array of knowledge** such as how far we are from the goal, path cost, how to reach to goal node, etc. *This knowledge help agents to explore less to the search space and find more efficiently the goal node.*

- The informed search algorithm is more useful for large search space.

- Informed search algorithm uses the idea of heuristic, so it is also called *Heuristic searc*h.

Heuristics function:

- Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.

- The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time.

- Heuristic function estimates how close a state is to the goal.

- It is represented by **h(n),** and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

$$h(n) <= h^*(n$$

Here h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

Pure Heuristic Search:

- Pure heuristic search is the simplest form of heuristic search algorithms. It expands nodes based on their heuristic value h(n). It maintains two lists, OPEN and CLOSED list. In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.

- On each iteration, each node n with the **lowest heuristic** value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues unit a goal state is found.

1. **Best First Search Algorithm(Greedy search)**

2. **A* Search Algorithm**

Best-first Search Algorithm (Greedy Search):

- Greedy best-first search algorithm always selects the path which appears best at that moment.

- It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search.

- Best-first search allows us to take the advantages of both algorithms. **With the help of best-first search, at each step, we can choose the most promising node.**

- In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function i.e

$$f(n)= g(n).$$

Best first search algorithm:

- **Step 1:** Place the starting node into the OPEN list.

- **Step 2:** If the OPEN list is empty, Stop and return failure.

- **Step 3:** Remove the node n, from the OPEN list which has the lowest value of h(n), and place it in the CLOSED list.

- **Step 4:** Expand the node n, and generate the successors of node n.

- **Step 5:** Check each successor of node n, and find whether any node is a goal node or not. If any successor node is the goal node, then return success and terminate the search, else proceed to Step 6.

- **Step 6:** For each successor node, the algorithm checks for evaluation function f(n), and then check if the node has been in either the OPEN or CLOSED list. If the node has not been in both lists, then add it to the OPEN list.

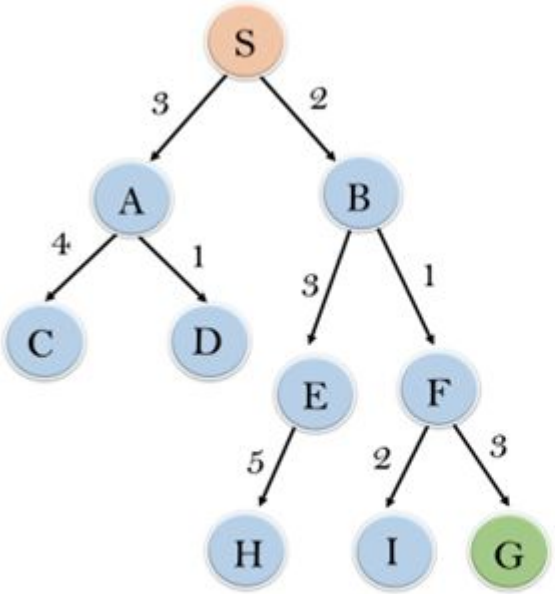
- **Step 7:** Return to Step 2.

Advantages:

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

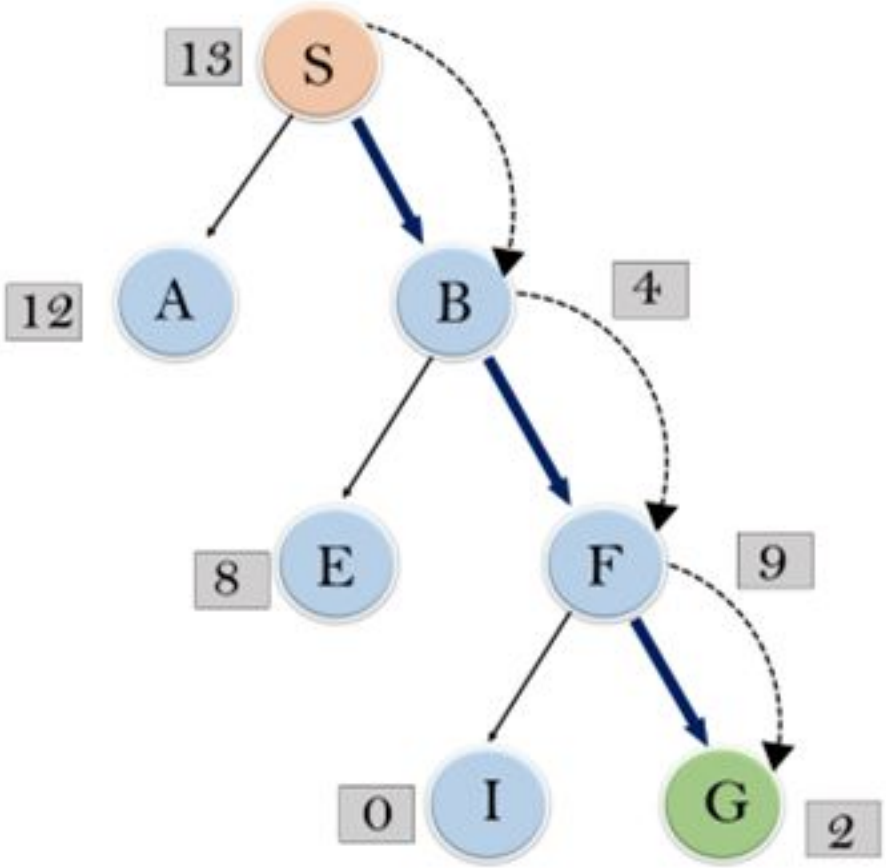
Disadvantages:

- It can behave as an unguided depth-first search in the worst-case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example


13
S
12
A
B
4
8
E
F
9
0
I
G
2

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]
: Open [E, A], Closed [S, B, F]

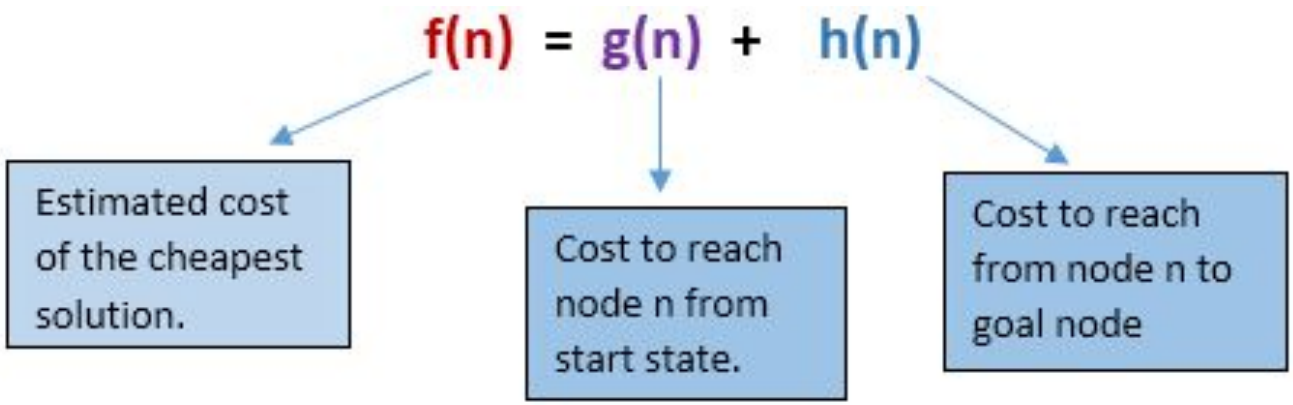
Iteration 3: Open [I, G, E, A], Closed [S, B, F]
: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be: S----> B----->F----> G

Time Complexity: The worst-case time complexity of Greedy best-first search is $O(b^m)$.
Space Complexity: The worst-case space complexity of Greedy best-first search is $O(b^m)$. Where m is the maximum depth of the search space.
Complete: Greedy best-first search is also incomplete, even if the given state space is finite.
Optimal: Greedy best-first search algorithm is not optimal.

A* Search Algorithm:

- A* search is the most commonly known form of best-first search.
- It uses heuristic function h(n), and cost to reach the node n from the start state g(n).
- It has combined features of UCS and greedy best-first search, by which it solves the problem efficiently.
- A* search algorithm finds the shortest path through the search space using the heuristic function.
- This search algorithm expands less search tree and provides optimal result faster.
- A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).



In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a **fitness number**.

Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

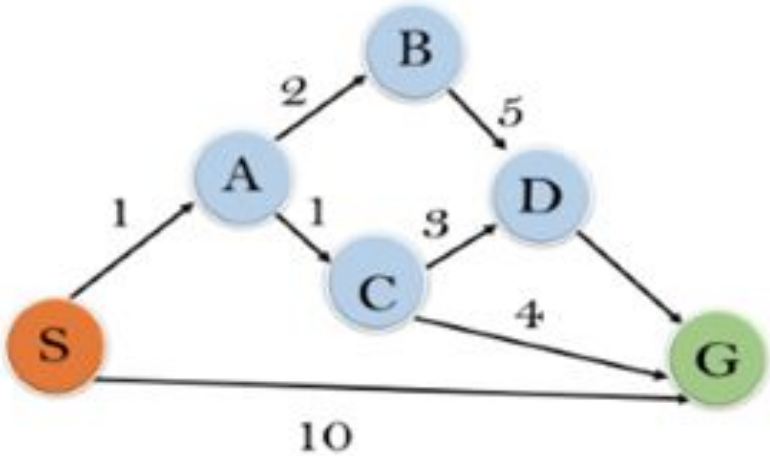
Step 6: Return to **Step 2**.

Advantages:

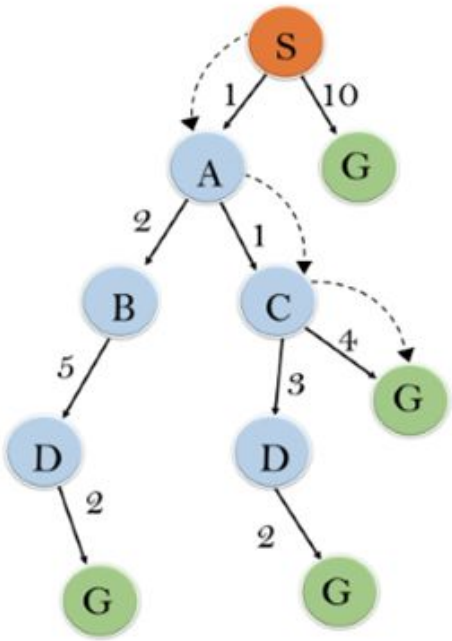
- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0


S
1
10
A
G
2
1
B
C
4
5
3
G
D
D
2
2
G
G

Initialization: {(S, 5)}
Iteration1: {(S--> A, 4), (S-->G, 10)}
Iteration2: {(S--> A-->C, 4), (S--> A-->B, 7), (S-->G, 10)}
Iteration3: {(S--> A-->C--->G, 6), (S--> A-->C--->D, 11), (S--> A-->B, 7), (S-->G, 10)}
Iteration 4 will give the final result, as **S--->A--->C--->G** it provides the optimal path with cost 6.

Points to remember:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition f(n)

Complete: A* algorithm is complete as long as:

- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

- **Admissible:** the first condition requires for optimality is that h(n) should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.

Time Complexity: The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d. So the time complexity is O(b^d), where b is the branching factor.

Space Complexity: The space complexity of A* search algorithm is **O(b^d)**

Admissible heuristic

A heuristic function is said to be **admissible** if it never overestimates the cost of reaching the goal, i.e. the cost it estimates to reach the goal is not higher than the lowest possible cost from the current point in the path

n is a node

h is a heuristic

$h(n)$ is cost indicated by h to reach a goal from n

$h^*(n)$ is the optimal cost to reach a goal from n

$h(n)$ is admissible if, $\forall n$

$$h(n) \leq h^*(n)$$

Consistent heuristic

a heuristic function is said to be **consistent, or monotone**, if its estimate is always less than or equal to the estimated distance from any neighboring vertex to the goal, plus the cost of reaching that neighbor.

Formally, for every node N and each successor P of N, the estimated cost of reaching the goal from N is no greater than the step cost of getting to P plus the estimated cost of reaching the goal from P. That is:

$$h(N) \leq c(N, P) + h(P) \text{ and}$$
$$h(G) = 0.$$

where

- *h* is the consistent heuristic function
- *N* is any node in the graph
- *P* is any descendant of *N*
- *G* is any goal node
- c(N, P) is the cost of reaching node P from N

Informally, every node *i* will give an estimate that, accounting for the cost to reach the next node, is always lesser than the estimate at node *i+1*.

A consistent heuristic is also admissible, i.e. it never overestimates the cost of reaching the goal (the converse, however, is not always true). This can be easily proved by **induction.**

Let $h(N_0) = 0$ be the estimated cost for the goal node. This implies that the base condition is trivially true as $0 \leq 0$. Since the heuristic is consistent,

$$h(N_{i+1}) \leq c(N_{i+1}, N_i) + h(N_i) \leq c(N_{i+1}, N_i) + c(N_i, N_{i-1}) + h(N_{i-1}) \leq c(N_{i+1}, N_i) + c(N_i, N_{i-1}) + \ldots + c(N_1, N_0) + h(N_0)$$

. The given terms are equal to the true cost, $\sum_{i=1}^{n} c(N_i, N_{i-1})$, so any consistent heuristic is also admissible since it is upperbounded by the true cost.